

Python Deep Dive — 30 Questions

Functions • Lambda • OOP • Datatypes

Focused practice for final exam preparation. Each question includes code traces, tricky edge cases, and conceptual explanations.

⚙️ Part 1: Functions (Q1-Q10)

Q1. What is the output of the following code?

```
python

def greet(name, msg="Hello"):
    return f"{msg}, {name}!"

print(greet("Alice"))
print(greet("Bob", "Hi"))
print(greet(msg="Hey", name="Carol"))
```

Answer:

```
Hello, Alice!
Hi, Bob!
Hey, Carol!
```

Default arguments fill in when omitted; keyword arguments can be passed in any order.

Q2. What will this print, and why is it a common bug?

```
python

def add_item(item, lst=[]):
    lst.append(item)
    return lst

print(add_item(1))
print(add_item(2))
print(add_item(3))
```

Answer:

```
[1]
[1, 2]
[1, 2, 3]
```

The default `lst=[]` is created **once** when the function is defined, not on each call. All calls share the same list object. Fix with `lst=None` and `if lst is None: lst = []` inside.

Q3. What is the output?

```
python

def outer(x):
    def inner(y):
        return x + y
    return inner

add5 = outer(5)
print(add5(3))
print(add5(10))
```

Answer:

```
8
15
```

`inner` is a **closure** — it captures `x=5` from `outer`'s scope even after `outer` has returned.

Q4. What is the difference between these two calls?

```
python

def f(a, b, c):
    print(a, b, c)

args = [1, 2, 3]
kwargs = {"a": 10, "b": 20, "c": 30}

f(*args)
f(**kwargs)
```

Answer:

```
1 2 3
10 20 30
```

`*args` unpacks a list into positional arguments; `**kwargs` unpacks a dict into keyword arguments.

Q5. What is the output and what concept does this demonstrate?

```
python

def counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment

c = counter()
print(c())
print(c())
print(c())
```

Answer:

```
1
2
3
```

`nonlocal` allows `increment` to **modify** `count` in the enclosing scope. Without `nonlocal`, `count += 1` would raise `UnboundLocalError`.

Q6. What does this function return? Trace it.

```
python

def mystery(n):
    if n <= 1:
        return n
    return mystery(n - 1) + mystery(n - 2)

print(mystery(6))
```

Answer: `8` — this is the Fibonacci sequence. Call stack builds up as: `f(6) → f(5)+f(4) → ... → 8`.

Q7. What is the output?

```
python

def f(*args, **kwargs):
    print(type(args), args)
    print(type(kwargs), kwargs)

f(1, 2, 3, x=4, y=5)
```

Answer:

```
<class 'tuple'> (1, 2, 3)
<class 'dict'> {'x': 4, 'y': 5}
```

`*args` is always a `tuple`; `**kwargs` is always a `dict`.

Q8. What does `functools.wraps` do and why is it important?

```
python

import functools

def decorator(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@decorator
def my_func():
    """My docstring."""
    pass

print(my_func.__name__)
print(my_func.__doc__)
```

Answer:

```
my_func
My docstring.
```

Without `@functools.wraps(func)`, `__name__` would print `wrapper` and `__doc__` would be `None`. It preserves the original function's metadata.

Q9. What is the output and what is `yield` doing here?

```
python

def gen():
    for i in range(1, 4):
        yield i * i

g = gen()
print(next(g))
print(next(g))
print(list(g))
```

Answer:

```
1
4
[9]
```

`yield` makes the function a **generator** — it pauses and resumes. `list(g)` consumes the remaining values.

Q10. Identify the error and fix it.

```
python

def apply(func, value):
    return func(value)

result = apply(print, "hello")
print(result)
```

Answer: No error in `apply()`. But `print()` returns `None`, so `result = None` and the last line prints `None`. If the intent was to capture output, use `return value` in a custom function instead. Output:

```
hello
None
```

λ Part 2: Lambda Functions (Q11-Q18)

Q11. What is the output?

python

```
square = lambda x: x ** 2
cube   = lambda x: x ** 3

ops = [square, cube]
for op in ops:
    print(op(3))
```

Answer:

9
27

Lambdas stored in a list and called like regular functions.

Q12. Sort the following list of dicts by `age` in descending order using a lambda.

python

```
people = [
    {"name": "Alice", "age": 30},
    {"name": "Bob", "age": 25},
    {"name": "Carol", "age": 35},
]
people.sort(key=lambda p: p["age"], reverse=True)
print([p["name"] for p in people])
```

Answer: `['Carol', 'Alice', 'Bob']`

Q13. What is the late-binding problem? What is the output?

python

```
funcs = [lambda x: x * i for i in range(1, 4)]
print([f(10) for f in funcs])
```

Answer: `[30, 30, 30]` All lambdas capture the **same variable** `i`, which is `3` after the loop. They don't capture the *value* of `i` at the time of creation.

Q14. Fix the late-binding bug from Q13.

python

```
funcs = [lambda x, i=i: x * i for i in range(1, 4)]
print([f(10) for f in funcs])
```

Answer: `[10, 20, 30]` Using `i=i` as a default argument captures the **current value** of `i` at each iteration.

Q15. Use `map()` and a lambda to convert Celsius to Fahrenheit.

```
python

celsius = [0, 20, 37, 100]
fahrenheit = list(map(lambda c: (c * 9/5) + 32, celsius))
print(fahrenheit)
```

Answer: `[32.0, 68.0, 98.6, 212.0]`

Q16. Use `filter()` and a lambda to keep only strings longer than 3 characters.

```
python

words = ["hi", "hello", "ok", "world", "no", "python"]
result = list(filter(lambda w: len(w) > 3, words))
print(result)
```

Answer: `['hello', 'world', 'python']`

Q17. What is the output?

```
python

from functools import reduce

nums = [1, 2, 3, 4, 5]
total = reduce(lambda acc, x: acc + x, nums, 10)
print(total)
```

Answer: `25` `reduce` folds left: `10 → 11 → 13 → 16 → 20 → 25`. The third argument `10` is the initial accumulator.

Q18. Can a lambda call another lambda? What is the output?

python

```
add = lambda x, y: x + y
double_sum = lambda x, y: add(x, y) * 2
print(double_sum(3, 4))
```

Answer: 14 Yes — lambdas can reference other lambdas or any callable in scope.

Part 3: OOP (Q19-Q26)

Q19. What is the output?

python

```
class Animal:
    kind = "animal"

    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound"

class Dog(Animal):
    def speak(self):
        return f"{self.name} barks"

d = Dog("Rex")
print(d.speak())
print(d.kind)
print(isinstance(d, Animal))
```

Answer:

```
Rex barks
animal
True
```

Q20. What is the output and what concept does it show?

python


```
class Counter:
    count = 0

    def __init__(self):
        Counter.count += 1

c1 = Counter()
c2 = Counter()
c3 = Counter()
print(Counter.count)
print(c1.count)
```

Answer:

3
3

`count` is a **class attribute** shared across all instances. Accessing it via an instance also works unless the instance shadows it.

Q21. What is the output of this MRO example?

```
python

class A:
    def hello(self): print("A")

class B(A):
    def hello(self): print("B")

class C(A):
    def hello(self): print("C")

class D(B, C):
    pass

D().hello()
print([cls.__name__ for cls in D.__mro__])
```

Answer:

B
['D', 'B', 'C', 'A', 'object']

Q22. What is the output? Trace `super()` carefully.

```
python

class Base:
    def __init__(self):
        print("Base __init__")

class Child(Base):
    def __init__(self):
        super().__init__()
        print("Child __init__")

class GrandChild(Child):
    def __init__(self):
        super().__init__()
        print("GrandChild __init__")

GrandChild()
```

Answer:

```
Base __init__
Child __init__
GrandChild __init__
```

`super()` chains up the MRO. `Base` runs first because `GrandChild → Child → Base`.

Q23. What are `@staticmethod` and `@classmethod`? Show the difference.

```
python

class MathUtils:
    multiplier = 2

    @staticmethod
    def add(a, b):
        return a + b

    @classmethod
    def scale(cls, value):
        return value * cls.multiplier

print(MathUtils.add(3, 4))
print(MathUtils.scale(5))
```

Answer:

```
7
10
```

`@staticmethod` — no access to class or instance. `@classmethod` — receives `cls`, can access class attributes.

Q24. Implement `__str__`, `__repr__`, and `__eq__` for a `Point` class.

python

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        return f"Point({self.x}, {self.y})"

    def __str__(self):
        return f"({self.x}, {self.y})"

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

p1 = Point(1, 2)
p2 = Point(1, 2)
print(repr(p1))    # Point(1, 2)
print(str(p1))     # (1, 2)
print(p1 == p2)    # True
```

Answer:

```
Point(1, 2)
(1, 2)
True
```

Q25. What is the output and what is `@property` doing?

python

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):
        return self._radius

    @radius.setter
    def radius(self, value):
        if value < 0:
            raise ValueError("Radius cannot be negative")
        self._radius = value

    @property
    def area(self):
        import math
        return round(math.pi * self._radius ** 2, 2)

c = Circle(5)
print(c.radius)
print(c.area)
c.radius = 10
print(c.area)
```

Answer:

```
5
78.54
314.16
```

`@property` makes methods accessible like attributes. The setter validates before assigning.

Q26. What is the output? What is name mangling?

python

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # private

    def get_balance(self):
        return self.__balance

acc = BankAccount(1000)
print(acc.get_balance())
# print(acc.__balance)           # AttributeError
print(acc._BankAccount__balance) # name mangling
```

Answer:

```
1000
1000
```

`__balance` is mangled to `_BankAccount__balance`. It's not truly private but harder to access accidentally.

Part 4: Datatypes (Q27-Q30)

Q27. What is the output? Explain the difference between shallow and deep copy.

```
python

import copy

original = [[1, 2], [3, 4]]
shallow  = copy.copy(original)
deep     = copy.deepcopy(original)

original[0].append(99)

print(shallow)    # affected?
print(deep)       # affected?
```

Answer:

```
[[1, 2, 99], [3, 4]]
[[1, 2], [3, 4]]
```

Shallow copy copies the outer list but the inner lists are still references. Deep copy recursively copies all nested objects.

Q28. What is the output of these dict operations?

python

```
d = {"a": 1, "b": 2, "c": 3}
```

```
# Pop with default
```

```
print(d.pop("z", 0))
```

```
# Merge (Python 3.9+)
```

```
extra = {"d": 4, "a": 99}
```

```
merged = d | extra
```

```
print(merged)
```

```
print(d)
```

Answer:

```
0
```

```
{'a': 99, 'b': 2, 'c': 3, 'd': 4}
```

```
{'a': 1, 'b': 2, 'c': 3}
```

| creates a **new** dict; d is unchanged. Later keys win on conflict.

Q29. What is the output? Explain tuple unpacking and (*) in unpacking.

python

```
first, *middle, last = [10, 20, 30, 40, 50]
```

```
print(first)
```

```
print(middle)
```

```
print(last)
```

```
a, b, *_ = (1, 2, 3, 4, 5)
```

```
print(a, b)
```

Answer:

```
10
```

```
[20, 30, 40]
```

```
50
```

```
1 2
```

*middle captures all elements between first and last. * is a convention for "don't care about the rest."

Q30. What is the output? What does this reveal about set operations?

```
python

A = {1, 2, 3, 4, 5}
B = {3, 4, 5, 6, 7}

print(A & B)      # intersection
print(A | B)      # union
print(A - B)      # difference
print(A ^ B)      # symmetric difference
print(A.issubset({1, 2, 3, 4, 5, 6}))
```

Answer:

```
{3, 4, 5}
{1, 2, 3, 4, 5, 6, 7}
{1, 2}
{1, 2, 6, 7}
True
```

Summary Table

Part	Questions	Key Concepts
⚙️ Functions	Q1-Q10	Defaults, closures, <code>*args</code> / <code>**kwargs</code> , <code>nonlocal</code> , recursion, generators
λ Lambda	Q11-Q18	<code>map</code> / <code>filter</code> / <code>reduce</code> , late binding, default capture fix
🏠 OOP	Q19-Q26	Inheritance, MRO, <code>super()</code> , <code>@property</code> , dunder methods, name mangling
📦 Datatypes	Q27-Q30	Shallow/deep copy, dict merge, tuple unpacking, set algebra

- 💡 Pro Tips:
- Draw a call stack when tracing recursion.
 - Always ask: *is this object mutable or immutable?*
 - For OOP questions, write out the MRO before answering method resolution questions.
 - Test lambda late-binding edge cases — they appear frequently in exams.

Best of luck! 🎓